

CHƯƠNG 1. GIỚI THIỆU

1.1. Giới thiệu bài báo

- Biểu đồ là một nguồn thông tin quan trọng, đóng vai trò cốt yếu trong việc trình bày và giải thích các mối quan hệ dữ liệu phức tạp. Chúng đã trở thành một phần không thể thiếu trong truyền thông thông tin, ra quyết định kinh doanh và nghiên cứu học thuật.
- Gần đây, các mô hình ngôn ngữ lớn đa phương thức (Multimodal Large Language Models - MLLMs) đã thể hiện năng lực mạnh mẽ và cho thấy khả năng vượt trội trong các nhiệm vụ hiểu biểu đồ khác nhau. Dựa trên nền tảng này, một số công trình đã phát triển các mô hình hiểu biểu đồ chuyên biệt thông qua việc tinh chỉnh có giám sát trên các bộ dữ liệu đa dạng.

1.2. Mô tả bài toán: Hiểu Biểu Đồ (Chart Understanding)

- Với sự gia tăng nhanh chóng của dữ liệu đa phương thức, việc tự động hiểu và diễn giải biểu đồ đã trở thành một nhu cầu cấp thiết. Bài toán Hiểu Biểu Đồ (Chart Understanding) yêu cầu mô hình có khả năng nắm bắt nội dung trong biểu đồ và hoàn thành các tác vụ được chỉ định thông qua hướng dẫn.
- Các tác vụ này bao gồm cả nhận dạng mức thấp (như trích xuất dữ liệu) và các tác vụ mức cao hơn như hỏi đáp (QA), tóm tắt nội dung, hay tái dựng biểu đồ. Do biểu đồ thường chứa nhiều văn bản OCR then chốt và các yêu cầu tính toán số học, bài toán này đòi hỏi khả năng nhận dạng văn bản mạnh mẽ và lý luận tính toán chính xác.
- **Bài toán cần giải quyết:**
 - **Đầu vào:** Một tập dữ liệu bao gồm các hình ảnh biểu đồ (biểu đồ cột, biểu đồ đường,...) và các câu hỏi ngôn ngữ tự nhiên (ví dụ: “Giá trị lớn nhất là bao nhiêu”).
 - **Đầu ra:** Mô hình TinyChart có thể trả lời các câu hỏi và vẽ lại biểu đồ.

1.3. Thực trạng, khó khăn và thách thức

- **Thực trạng:** Các mô hình MLLM hiểu biểu đồ gần đây chủ yếu chuyển hướng sang phương pháp end-to-end, kế thừa cấu trúc của MLLM và tăng cường khả năng thông qua

việc tinh chỉnh có giám sát (supervised fine-tuning) trên các bộ dữ liệu hướng dẫn quy mô lớn .

- Mặc dù các mô hình này đạt được những cải thiện rõ rệt , chúng vẫn đối mặt với nhiều hạn chế . Hạn chế lớn và phổ biến nhất là **kích thước mô hình khổng lồ** . Ví dụ, các mô hình như ChartLlama có quy mô lên tới 13 tỷ tham số , khiến việc huấn luyện và triển khai trở nên khó khăn, đặc biệt trong các môi trường có nguồn lực tính toán hạn chế (ví dụ, yêu cầu VRAM GPU lớn) .
- **Khó khăn và Thách thức chính:** Từ thực trạng trên, có ba thách thức kỹ thuật cốt lõi mà các MLLM hiệu biểu đồ đang đối mặt:
 - **Kích thước mô hình và chi phí triển khai:** Kích thước tuyệt đối của các mô hình này về số lượng tham số và yêu cầu tính toán đã hạn chế việc sử dụng chúng trong các môi trường có nguồn lực hạn chế .
 - **Năng lực suy luận và tính toán số học:** Các mô hình hiện tại thể hiện khả năng còn yếu trong việc xử lý các câu hỏi liên quan đến **phép tính số học** . Chúng gặp khó khăn trong việc trả lời trực tiếp các câu hỏi này, vốn đòi hỏi suy luận logic nhiều bước . Thống kê trên bộ dữ liệu ChartQA cho thấy có tới 42% câu hỏi yêu cầu câu trả lời dạng số .
 - **Mã hóa hình ảnh độ phân giải cao:** Hiệu quả mã hóa hình ảnh có độ phân giải cao chưa được tối ưu . Việc hiểu biểu đồ đòi hỏi mô hình phải nhận dạng được các chi tiết nhỏ và văn bản OCR, điều này thường yêu cầu đầu vào có độ phân giải cao . Tuy nhiên, bộ biến đổi thị giác (vision transformer) tiêu chuẩn sẽ tạo ra **chuỗi đặc trưng thị giác quá dài**, gây tổn kém tài nguyên tính toán và bộ nhớ .

1.4. Bối cảnh và động lực nghiên cứu

- Nhu cầu cấp thiết về việc tự động hiểu biểu đồ cùng những hạn chế lớn của các mô hình hiện tại là động lực chính của nghiên cứu này. Để khắc phục những hạn chế về kích thước mô hình, khả năng tính toán số học và hiệu quả mã hóa ảnh , bài báo này giới thiệu TinyChart —một MLLM hiệu quả (chỉ với 3 tỷ tham số) và mạnh mẽ dành cho bài toán hiểu biểu đồ.

1.5. Giới thiệu tác giả và tổ chức

Bài báo TinyChart được thực hiện bởi các tác giả đến từ Đại học Nhân dân Trung Quốc và Tập đoàn Alibaba (Alibaba Group) trong đó có 3 tác giả chính:

1.5.1. Liang Zhang

- Liang Zhang là nghiên cứu sinh tiến sĩ tại Đại học Nhân dân Trung Quốc. Lĩnh vực nghiên cứu của ông tập trung vào Học Đa phương thức (Multimodal Learning), Hiểu Tài liệu (Document Understanding) (đặc biệt là các phương pháp không cần OCR), và Hiểu Biểu đồ (Chart Understanding). Ông cũng nghiên cứu về việc đánh giá chú thích hình ảnh (image caption evaluation) và truy xuất đa phương thức (cross-modal retrieval).
- Công bố của tác giả trên các nền tảng:
 - Google Scholar: <https://scholar.google.com/citations?user=Tqdm1-oAAAAJ>
- Tổng số trích dẫn: 1.967
- H-index: 12
- i10-index: 13

1.5.2. Ming Yan

- Anwen Hu là nhà nghiên cứu tại Tập đoàn Alibaba (Alibaba Group), và đã hoàn thành chương trình tiến sĩ tại Đại học Nhân dân Trung Quốc. Ông là một thành viên chủ chốt trong nhóm phát triển các mô hình đa phương thức mPLUG. Lĩnh vực chuyên môn của ông là Mô hình Ngôn ngữ-Thị giác Lớn (LVLMs), Hiểu Tài liệu (Document Understanding), và Chú thích Hình ảnh có kiểm soát (Controlled Image Captioning).
- Công bố của tác giả trên các nền tảng:
 - Google Scholar: <https://scholar.google.com/citations?user=FqvDzH8AAAAJ>

1.5.3. Anwen Hu

- Haiyang Xu là nhà nghiên cứu tại Tập đoàn Alibaba, tập trung vào các mô hình nền tảng đa phương thức. Ông là tác giả quan trọng trong nhiều dự án mPLUG, bao gồm mPLUG-2, mPLUG-Owl3, và các mô hình hiểu chuỗi hình ảnh dài (long image-sequence understanding). Lĩnh vực của ông bao gồm Mô hình Đa phương thức (Multimodal Models) và Học Đa phương thức (Multimodality).
- Công bố của tác giả trên các nền tảng:
 - Google Scholar: <https://scholar.google.com/citations?user=qZYvce8AAAAJ>

CHƯƠNG 2. KIẾN THỨC NỀN TẢNG

2.1. Mô hình Ngôn ngữ lớn Đa phương thức (MLLM)

Mô hình Ngôn ngữ lớn Đa phương thức (Multimodal Large Language Models - MLLM) đại diện cho một bước tiến hóa quan trọng từ các Mô hình Ngôn ngữ lớn (LLM) truyền thống. Trong khi LLM chỉ xử lý và tạo ra văn bản, MLLM được thiết kế để tiếp nhận, xử lý và suy luận đồng thời trên nhiều luồng thông tin (phương thức) khác nhau, phổ biến nhất là văn bản và hình ảnh.

Khả năng cốt lõi của MLLM là sự hội tụ của tri giác thị giác (visual perception) và khả năng suy luận ngôn ngữ (linguistic reasoning). Các mô hình này không chỉ "nhìn" một hình ảnh và "đọc" một câu hỏi; chúng có khả năng *hiểu* mối quan hệ sâu sắc giữa các yếu tố trong hình ảnh và các khái niệm được mô tả trong văn bản, từ đó tạo ra các phản hồi phức tạp, có bối cảnh và chính xác.

2.1.1. Kiến trúc chung (Encoder, Connector, LLM)

Mặc dù có nhiều biến thể, kiến trúc của MLLM hiện đại thường tuân theo một mô-đun ba thành phần cơ bản, được thiết kế để cho phép dòng thông tin chảy từ phương thức thị giác sang phương thức ngôn ngữ:

- Bộ mã hóa thị giác (Visual Encoder): Đây là thành phần đầu vào cho dữ liệu hình ảnh. Thường là một mô hình Vision Transformer (ViT) hoặc một biến thể đã được tiền huấn luyện, nhiệm vụ của nó là "nhìn" ảnh thô (raw pixels) và chuyển đổi nó thành một tập hợp các đặc trưng thị giác (visual features) ở dạng véc-tơ. Các véc-tơ này biểu diễn các thông tin trích xuất từ hình ảnh, chẳng hạn như vật thể, kết cấu, và văn bản.
- Bộ kết nối (Connector Module): Thành phần này đóng vai trò then chốt như một "cầu nối" hoặc "bộ phiên dịch" giữa hai miền. Bộ mã hóa thị giác và LLM vốn "nói" hai "ngôn ngữ" khác nhau (không gian đặc trưng của chúng không tương thích). Bộ kết nối, thường là một mạng nơ-ron nhỏ (như MLP) hoặc một cơ chế chú ý chéo (cross-attention), có nhiệm vụ ánh xạ (map) các véc-tơ đặc trưng thị giác vào không gian nhúng (embedding space) mà LLM có thể hiểu được.
- Mô hình Ngôn ngữ lớn (Large Language Model - LLM): Đây là "bộ não" trung tâm của hệ thống. LLM nhận đầu vào là một chuỗi kết hợp bao gồm cả các hướng dẫn bằng văn

bản (text prompt) và các đặc trưng thị giác đã được "phiên dịch" từ bộ kết nối. Dựa trên thông tin đa phương thức này, LLM thực hiện quá trình suy luận phức tạp, hiểu bối cảnh và tạo ra phản hồi cuối cùng dưới dạng văn bản.

2.1.2. Huấn luyện MLLM

Quá trình huấn luyện MLLM là một quy trình phức tạp, thường được chia thành hai giai đoạn chính.

Đầu tiên là giai đoạn Tiền huấn luyện Căn chỉnh (Alignment Pre-training), [Hình ảnh mạng nơ-ron kết nối hình ảnh và văn bản] mô hình được "dạy" cách liên kết các khái niệm cơ bản giữa hai phương thức. Nó học trên hàng tỷ cặp (hình ảnh, chú thích) để hiểu rằng một tập hợp pixel cụ thể (ví dụ: hình ảnh một con chó) tương ứng với một khái niệm ngôn ngữ (từ "chó").

Giai đoạn thứ hai là Tinh chỉnh theo Hướng dẫn (Instruction Fine-tuning). Sau khi đã "biết" liên kết hình ảnh và văn bản, mô hình được "dạy" cách "làm" theo mệnh lệnh. Nó được huấn luyện trên một bộ dữ liệu hỏi-đáp chất lượng cao, giúp mô hình học cách tuân theo các hướng dẫn cụ thể và trở thành một trợ lý hữu ích (ví dụ: trả lời câu hỏi, mô tả chi tiết về hình ảnh).

2.2. Các phương pháp Hiểu Biểu Đồ

Hiểu biểu đồ (Chart Understanding) là một lĩnh vực ứng dụng chuyên biệt và đầy thách thức của MLLM. Nhiệm vụ này đòi hỏi năng lực vượt xa nhận dạng hình ảnh thông thường; nó yêu cầu mô hình phải thực hiện đồng thời nhiều kỹ năng phức tạp: nhận dạng ký tự quang học (OCR) để đọc nhãn và số liệu, phân tích cấu trúc để hiểu loại biểu đồ (tròn, cột, đường), và suy luận logic-toán học để so sánh các giá trị, xác định xu hướng hoặc tính toán các kết quả mới.

Trong lịch sử, việc giải quyết bài toán này phân chia thành hai hướng tiếp cận chính: phương pháp dựa trên pipeline và phương pháp end-to-end.

2.2.1. Phương pháp dựa trên Pipeline

Đây là phương pháp truyền thống, tiếp cận bài toán bằng cách chia nhỏ thành một chuỗi các bước xử lý độc lập:

- Trích xuất có cấu trúc: Ở bước đầu tiên, hình ảnh biểu đồ được đưa qua các công cụ chuyên dụng, chẳng hạn như một hệ thống OCR để đọc tất cả văn bản và một bộ phát hiện thành phần (component detector) để xác định các yếu tố đồ họa (ví dụ: vị trí các cột, tọa độ các điểm trên đường).
- Chuyển đổi dữ liệu: Dữ liệu trích xuất từ bước 1 (văn bản, tọa độ) được biên dịch thành một dạng biểu diễn có cấu trúc trung gian, chẳng hạn như một bảng dữ liệu (data table) hoặc một file JSON.
- Suy luận ngôn ngữ: Cuối cùng, bảng dữ liệu này (chứ không phải hình ảnh gốc) được cung cấp cho một Mô hình Ngôn ngữ (LLM) thuần túy để trả lời câu hỏi hoặc tóm tắt.

Hạn chế lớn nhất của phương pháp này là lỗi tích lũy (error accumulation). Bất kỳ sai sót nào ở bước 1 (ví dụ: OCR đọc sai một con số) sẽ không thể được sửa chữa và sẽ lan truyền, dẫn đến kết quả cuối cùng hoàn toàn sai lệch. Hơn nữa, mô hình ngôn ngữ ở bước 3 bị "mù" (không thấy) hoàn toàn hình ảnh gốc, do đó không có khả năng tự kiểm chứng thông tin.

2.2.2. Phương pháp End-to-End

Các MLLM hiện đại cho phép một cách tiếp cận tiên tiến hơn là "từ đầu đến cuối" (end-to-end).

Trong mô hình này, không có các bước trung gian độc lập:

- Xử lý hợp nhất: MLLM nhận trực tiếp hình ảnh biểu đồ thô và câu hỏi bằng văn bản làm đầu vào.
- Học tập đồng thời: Toàn bộ hệ thống, từ bộ mã hóa thị giác đến LLM, được huấn luyện chung (jointly optimized) để thực hiện đồng thời tất cả các nhiệm vụ. Mô hình tự học cách đọc văn bản, phân tích cấu trúc đồ họa và thực hiện suy luận logic trong một mạng nơ-ron duy nhất.

Phương pháp end-to-end về lý thuyết vượt trội hơn vì nó cho phép mô hình học được các mối tương quan phức tạp giữa tri giác thị giác và suy luận. Tuy nhiên, nó cũng đặt ra những thách thức mới: nó đòi hỏi các mô hình có kích thước tham số khổng lồ, yêu cầu tài nguyên tính toán cực lớn (như đã thảo luận ở mục 2.3), và đặc biệt vẫn vật lộn với việc đảm bảo tính chính xác tuyệt đối trong các phép tính số học (như sẽ thảo luận ở mục 2.4).

2.3. Thách thức trong việc Xử lý hình ảnh độ phân giải cao trong MLLM

Hiệu suất của các Mô hình Ngôn ngữ lớn Đa phương thức (MLLM) trong các tác vụ chuyên biệt như hiểu tài liệu và biểu đồ thường xuyên bị hạn chế. Nguyên nhân chính xuất phát từ việc nhiều kiến trúc MLLM tiêu chuẩn sử dụng độ phân giải đầu vào thấp nhằm tối ưu hóa chi phí tính toán. Tuy nhiên, hành động co nhỏ hình ảnh này dẫn đến việc làm mất đi các thông tin then chốt. Đối với biểu đồ, các chi tiết quan trọng như văn bản (OCR), nhãn trên các trục tọa độ, hoặc các điểm dữ liệu nhỏ lẻ có thể trở nên không thể nhận dạng được. Do đó, việc xử lý hình ảnh đầu vào ở độ phân giải cao được xem là một yêu cầu thiết yếu để đạt được khả năng hiểu chi tiết và chính xác. Mặc dù vậy, việc tăng độ phân giải đầu vào một cách trực tiếp sẽ tạo ra một thách thức tính toán vô cùng lớn, bắt nguồn từ kiến trúc cốt lõi của các bộ mã hóa thị giác (Visual Encoders). Các bộ mã hóa này, điển hình là Vision Transformer (ViT), hoạt động bằng cách chia hình ảnh thành một lưới các "mảnh" (patches) nhỏ. Các mảnh này sau đó được tuyến tính hóa thành một chuỗi đặc trưng thị giác (visual feature sequence) và đưa vào Mô hình Ngôn ngữ (LLM).

Vấn đề mấu chốt nằm ở chỗ độ dài của chuỗi này - một yếu tố quyết định gánh nặng tính toán và bộ nhớ - tăng theo tỷ lệ bình phương (quadratically) với độ phân giải. Ví dụ, việc tăng gấp đôi độ phân giải (ví dụ: từ 384x384 lên 768x768) sẽ làm tăng độ dài chuỗi đặc trưng lên gấp bốn lần.

Hậu quả trực tiếp là một "chuỗi đặc trưng thị giác quá dài" (excessively long visual sequence), đòi hỏi một lượng tài nguyên bộ nhớ (VRAM) khổng lồ, vượt quá khả năng của ngay cả các hệ thống GPU hiệu năng cao. Trong thực tế, điều này thường dẫn đến lỗi Hết bộ nhớ (Out-of-Memory - OOM), khiến cho việc huấn luyện và suy luận mô hình trở nên bất khả thi.

Tóm lại, tồn tại một mâu thuẫn cơ bản: MLLM cần độ phân giải cao để *hiểu* chính xác biểu đồ, nhưng các kiến trúc hiện tại lại không thể *xử lý* hiệu quả gánh nặng tính toán mà độ phân giải cao mang lại. Giải quyết bài toán cân bằng giữa độ chính xác về tri giác (perceptual accuracy) và tính khả thi về tính toán (computational feasibility) là một hướng nghiên cứu quan trọng trong lĩnh vực này.

2.4. Phương pháp Program-of-Thoughts (PoT)

Một trong những điểm yếu cố hữu của các Mô hình Ngôn ngữ lớn (LLM), và *κατ' επέκταση* là MLLM, là khả năng thực hiện các phép suy luận số học (numerical reasoning) chính xác. Khi đối mặt với các câu hỏi yêu cầu tính toán dựa trên dữ liệu trực quan (như trong biểu đồ), các mô hình này thường cố gắng tạo ra câu trả lời cuối cùng (ví dụ: một con số) một cách trực tiếp. Quá trình này dễ phát sinh lỗi vì mô hình phải đồng thời vừa trích xuất đúng dữ liệu, vừa hiểu đúng logic toán học, và vừa thực hiện phép tính một cách chính xác—tất cả trong một bước tạo văn bản duy nhất.

Để giải quyết vấn đề này, phương pháp "Chương trình Tư duy" (Program-of-Thoughts - PoT) đã được đề xuất như một chiến lược học tập thay thế.

Thay vì huấn luyện mô hình để tạo ra câu trả lời cuối cùng, phương pháp PoT huấn luyện mô hình tạo ra một đoạn mã (program) có thể thực thi được—chẳng hạn như mã Python—đại diện cho các bước logic cần thiết để giải quyết vấn đề. Câu trả lời cuối cùng sau đó không được tạo ra bởi mô hình, mà được thu thập bằng cách thực thi đoạn mã này thông qua một trình thông dịch (interpreter) tiêu chuẩn.

Ưu điểm cốt lõi của phương pháp PoT là sự **tách biệt rõ ràng giữa suy luận logic và tính toán số học** (decoupling reasoning from calculation). Mô hình ngôn ngữ chỉ cần tập trung vào nhiệm vụ mà nó làm tốt nhất: hiểu bối cảnh (từ hình ảnh và văn bản) và diễn giải nó thành một chuỗi các bước logic (chương trình). Gánh nặng của việc tính toán chính xác sau đó được chuyển giao cho một hệ thống bên ngoài vốn được thiết kế cho việc đó (trình thông dịch Python).

Hơn nữa, việc tạo mã cung cấp một tín hiệu học tập (learning signal) phong phú và có cấu trúc hơn nhiều so với một câu trả lời số học đơn lẻ. Một chương trình bao gồm các biến, các hàm, và các bình luận, mô phỏng một quá trình suy luận từng bước (multi-step reasoning). Điều này gần gũi hơn với dữ liệu tiền huấn luyện của LLM, vốn bao gồm một lượng lớn mã nguồn, do đó giúp mô hình học cách giải quyết các vấn đề phức tạp một cách hiệu quả và đáng tin cậy hơn.

CHƯƠNG 3. PHÂN TÍCH NỘI DUNG BÀI BÁO (TINYCHART)

Chương này đi sâu vào phân tích bài báo "TinyChart: Efficient Chart Understanding with Visual Token Merging and Program-of-Thoughts Learning" [1]. Đây là công trình đề xuất một giải pháp cụ thể cho các thách thức nền tảng đã được thảo luận tại Chương 2.

3.1. Chủ đề và Mục tiêu của Bài báo

Chủ đề trọng tâm của công trình này là phát triển một Mô hình Ngôn ngữ lớn Đa phương thức (MLLM) *hiệu quả* (efficient) chuyên biệt cho nhiệm vụ hiểu biểu đồ. Như đã phân tích ở Chương 2, MLLM truyền thống khi đối mặt với biểu đồ gặp phải hai thách thức lớn: (1) chi phí tính toán khổng lồ khi xử lý ảnh độ phân giải cao, vốn là yêu cầu bắt buộc để đọc chi tiết (Vấn đề 2.3), và (2) khả năng suy luận số học yếu, dẫn đến các lỗi logic-toán học (Vấn đề 2.4).

Bài báo giới thiệu **TinyChart** như một giải pháp cho cả hai vấn đề. Đây là một mô hình tương đối nhỏ (chỉ 3 tỷ tham số) nhưng được thiết kế để đạt được hiệu suất hàng đầu (SOTA), giải quyết mâu thuẫn giữa hiệu suất và chi phí tính toán.

3.2. Các Đóng góp Kỹ thuật chính

Để giải quyết các thách thức nêu trên, bài báo giới thiệu ba đóng góp kỹ thuật chính.

Thứ nhất, bài báo ra mắt **TinyChart**, một MLLM 3 tỷ tham số chứng minh được khả năng vượt trội so với các mô hình lớn hơn nhiều (ví dụ: 13 tỷ tham số) và thậm chí cả các mô hình đóng như GPT-4V trên các bộ benchmark chuyên biệt về biểu đồ, đồng thời duy trì tốc độ suy luận nhanh hơn đáng kể.

Thứ hai, để giải quyết Vấn đề 2.3 (xử lý ảnh nét), bài báo đề xuất áp dụng **Visual Token Merging (VTM)**. Đây là một mô-đun "Gộp Token Thị giác" được tích hợp vào bộ mã hóa, cho phép xử lý hiệu quả ảnh độ phân giải cao bằng cách giảm đáng kể độ dài chuỗi đặc trưng thị giác, từ đó tránh được lỗi hết bộ nhớ (OOM).

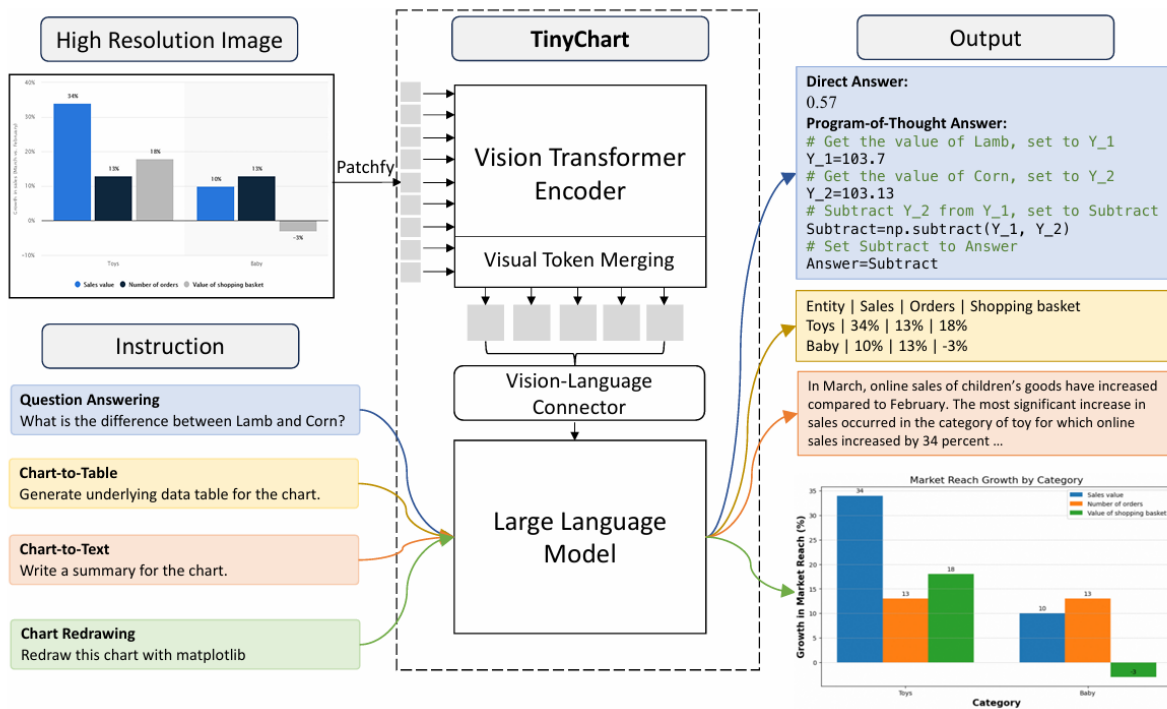
Thứ ba, để giải quyết Vấn đề 2.4 (suy luận số học yếu), bài báo đề xuất chiến lược huấn luyện **Program-of-Thoughts (PoT) Learning**. Kèm theo đó, tác giả xây dựng một bộ dữ liệu mới là

ChartQA-PoT để "dạy" mô hình cách tạo ra các chương trình Python thay vì tự tính toán, tách biệt khả năng suy luận logic khỏi việc thực thi tính toán.

3.3. Phân tích Kiến trúc và Phương pháp luận

3.3.1. Tổng quan Kiến trúc và Giải pháp VTM

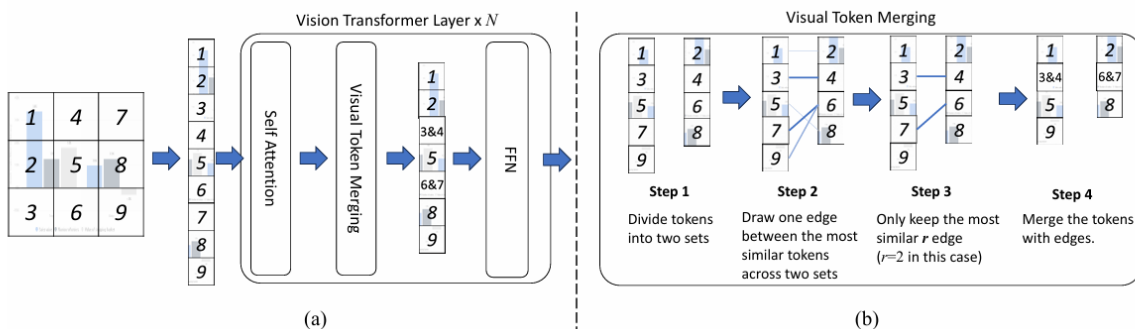
Kiến trúc của TinyChart (minh họa trong Hình 1) tuân theo nền tảng MLLM ba thành phần tiêu chuẩn (Mục 2.1.1), bao gồm một Bộ mã hóa thị giác (Visual Encoder), một Bộ kết nối (Connector) và một Mô hình Ngôn ngữ lớn (LLM). Cụ thể, mô hình sử dụng **SigLIP (384 triệu tham số)** làm bộ mã hóa và **Phi-2 (2.7 tỷ tham số)** làm LLM, được liên kết bởi một mạng MLP đơn giản.



Hình 1: Tổng quan về TinyChart.

Cải tiến kiến trúc then chốt của TinyChart là việc tích hợp mô-đun **Visual Token Merging (VTM)** vào *bên trong* mỗi lớp của bộ mã hóa thị giác (SigLIP). Như đã phân tích ở Mục 2.3, ảnh độ phân giải cao (ví dụ: 768x768) là cần thiết để đọc OCR, nhưng sẽ tạo ra một chuỗi token quá dài (ví dụ: 2916 token), gây lỗi OOM. Giải pháp VTM (minh họa trong Hình 2) hoạt động bằng

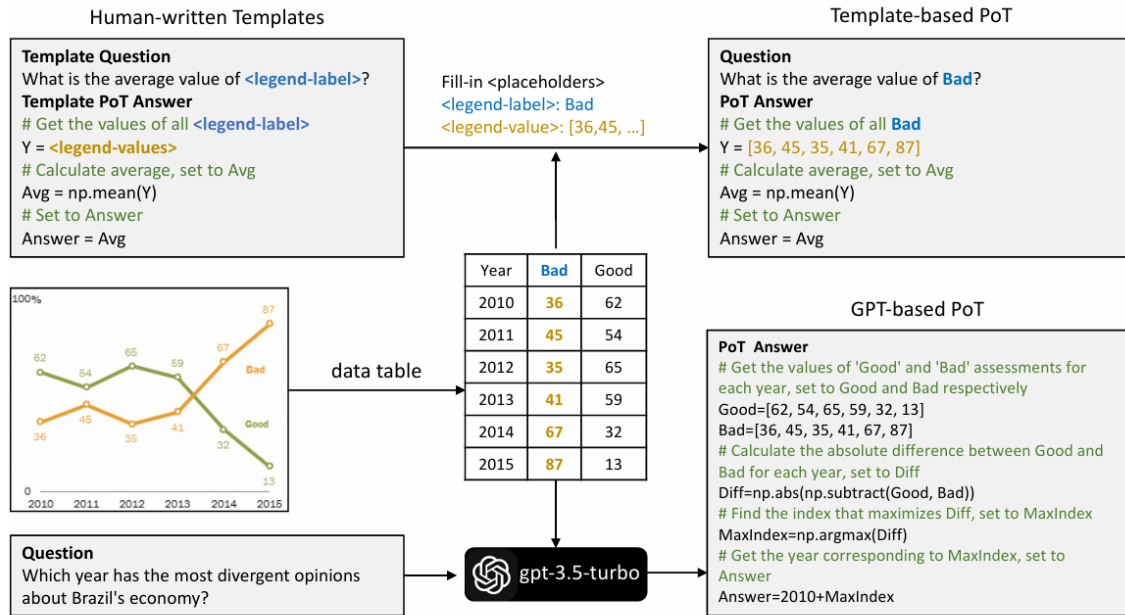
cách tự động xác định các token (vùng ảnh) giống nhau nhất về mặt đặc trưng (như nền trắng, mảng màu đồng nhất) và "gộp" chúng lại. Kết quả là chuỗi đặc trưng được giảm đáng kể (ví dụ: từ 2916 xuống 732 token), cho phép mô hình xử lý ảnh nét mà vẫn giữ được hiệu suất tính toán.



Hình 2: (a) Lớp transformer thị giác với hợp nhất token hình ảnh. (b) Quy trình hợp nhất token

3.3.2. Phương pháp học PoT và Bộ dữ liệu ChartQA-PoT

Song song với cải tiến kiến trúc là một chiến lược huấn luyện thông minh: **Program-of-Thoughts (PoT)**. Để giải quyết Vấn đề 2.4, thay vì bắt LLM (Phi-2) phải tự thực hiện các phép tính số học dễ lỗi, mô hình được huấn luyện để *tạo ra một chương trình Python* mô tả các bước suy luận. Đoạn mã này sau đó được thực thi bởi một trình thông dịch Python để trả về kết quả chính xác, qua đó tách biệt khả năng suy luận (LLM làm) khỏi khả năng tính toán (Python làm). Để hiện thực hóa phương pháp PoT, một bộ dữ liệu mới là **ChartQA-PoT** (gồm ~140K mẫu) đã được xây dựng (minh họa trong Hình 3). Bộ dữ liệu này được tạo bằng hai phương pháp: (1) **Template-based PoT** (~119K mẫu), tự động điền vào các mẫu mã (template) đơn giản, và (2) **GPT-based PoT** (~21K mẫu), sử dụng gpt-3.5-turbo để tạo ra các chuỗi suy luận phức tạp và đa dạng hơn.



Hình 3: Minh họa việc xây dựng PoT dựa trên Mẫu (nửa trên) và PoT dựa trên GPT (nửa dưới) trong bộ dữ liệu ChartQA PoT.

3.4. Cấu trúc Bài báo

Bài báo được tổ chức theo cấu trúc IMRAD (Giới thiệu, Phương pháp, Kết quả và Thảo luận) tiêu chuẩn của một công trình nghiên cứu khoa học:

- **Chương 1 (Introduction):** Giới thiệu bối cảnh, các vấn đề (chi phí cao, tính toán yếu) và tóm tắt các đóng góp.
- **Chương 2 (Related Work):** Điểm lại các công trình liên quan (tương ứng với Chương 2 của luận văn này).
- **Chương 3 (TinyChart):** Trình bày chi tiết về phương pháp luận, bao gồm kiến trúc, VTM, và PoT Learning.
- **Chương 4 (Experiment):** Mô tả chi tiết thực nghiệm, kết quả và các nghiên cứu loại bỏ (ablation studies).
- **Chương 5 (Conclusion):** Tóm tắt lại các đóng góp và kết quả đạt được.

3.5. Phân tích Thực nghiệm và Kết quả

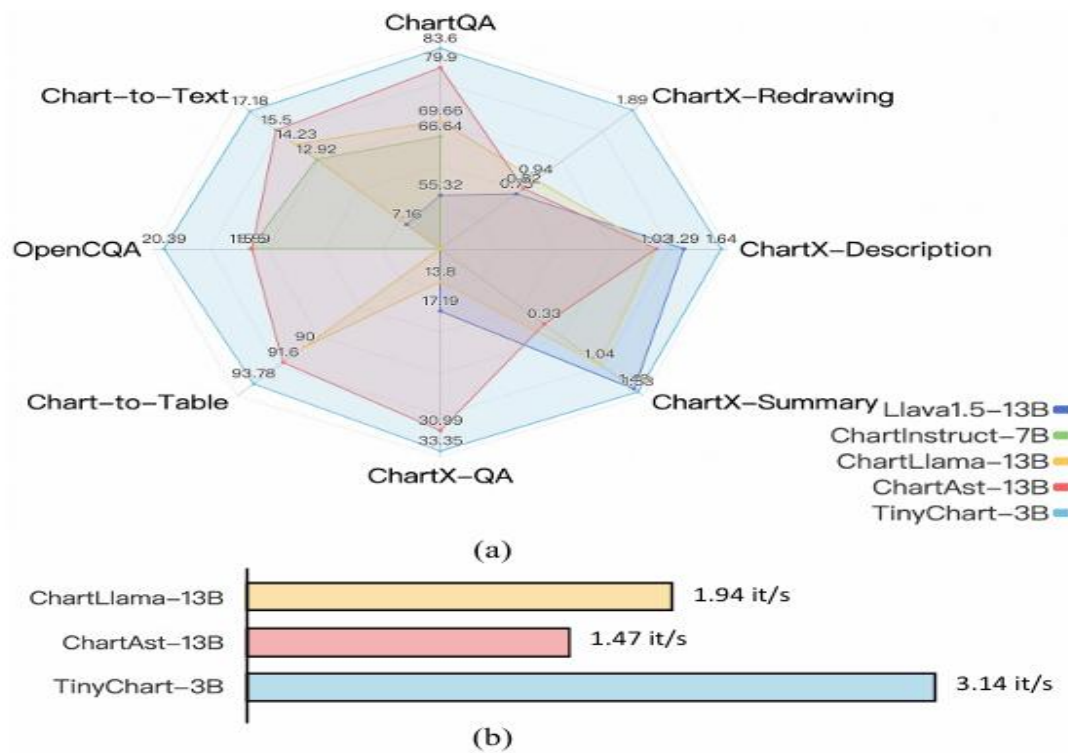
Phần thực nghiệm của bài báo cung cấp những bằng chứng định lượng mạnh mẽ về hiệu quả của các phương pháp đề xuất.

3.5.1. Thiết lập Thực nghiệm

Mô hình được huấn luyện đa tác vụ (Multitask Learning) trên một bộ dữ liệu hỗn hợp lớn (1.36 triệu mẫu, Bảng 1 trong [1]), bao gồm nhiều tác vụ như QA (ChartQA, PlotQA), tóm tắt (Chart-to-Text), và trích xuất (Chart-to-Table). Tác giả huấn luyện mô hình ở hai độ phân giải: 512x512 (với $r=20$ token gộp) và 768x768 (với $r=84$ token gộp). Việc đánh giá được thực hiện trên một loạt các bộ benchmark đa dạng như ChartQA, Chart-to-Text, Chart-to-Table, OpenCQA, và ChartX.

3.5.2. Kết quả chính và So sánh SOTA

Kết quả chính (Bảng 1 và Hình 4) cho thấy hiệu quả vượt trội của TinyChart. Mặc dù chỉ có 3 tỷ tham số, TinyChart (ở 768x768) đạt 83.60 điểm trên ChartQA, vượt qua các mô hình lớn hơn nhiều như ChartAst (13B, 79.90 điểm) và cả các mô hình đồng như GPT-4V (78.50 điểm). Đồng thời, tốc độ suy luận của nó (3.14 it/s) cũng nhanh hơn đáng kể so với ChartAst (1.47 it/s), chứng minh cho tính hiệu quả của kiến trúc.



Hình 4: TinyChart-3B vượt trội hơn một số MLLM 13B trên nhiều tiêu chuẩn hiệu biểu đồ (a), đồng thời đạt được thông lượng suy luận lớn hơn (b)

Dataset	Benchmark	Samples
<i>Chart question answer</i>		
ChartQA [32]	✓	28,299
ChartQA-PoT	✓	140,584
PlotQA [36]		157,070
DVQA [19]		200,000
OpenCQA [20]		5,407
<i>Chart-to-text generation</i>		
Pew [21]	✓	7,892
Statista [21]	✓	29,589
OpenCQA [20]		5,407
Vistext [45]		11,171
ChartSumm [42]		75,255
Chart2Text-8k [37]		7,862
<i>Chart-to-table generation</i>		
ChartQA [32]	✓	19,373
PlotQA [36]		190,720
Chart2Text-8k		8,305
DVQA [19]		300,000
Statista [21]		29,589
<i>Chart instruction following</i>		
ChartLlama [10]		148,398
Total		1,364,921

Bảng 1: Các bộ dữ liệu được sử dụng để huấn luyện TinyChart. Các bộ dữ liệu chuẩn bao gồm các đánh giá cơ bản về hiểu biết biểu đồ, bao gồm QA, tóm tắt và sinh bảng từ biểu đồ. (Lưu ý rằng trong các nghiên cứu loại bỏ, chúng tôi chỉ sử dụng các bộ dữ liệu chuẩn để huấn luyện do giới hạn về tài nguyên tính toán).

3.5.3. Phân tích Hiệu quả (Ablation Studies)

Nghiên cứu loại bỏ (Mục 4.4 trong bài báo và Bảng 2) làm rõ tầm quan trọng của từng đóng góp. Nghiên cứu về **Program-of-Thoughts (PoT)** (Bảng 2) cho thấy phương pháp này cải thiện đáng kể độ chính xác trên các câu hỏi tính toán (tăng từ 56.64% khi trả lời trực tiếp lên 78.98% khi dùng PoT).

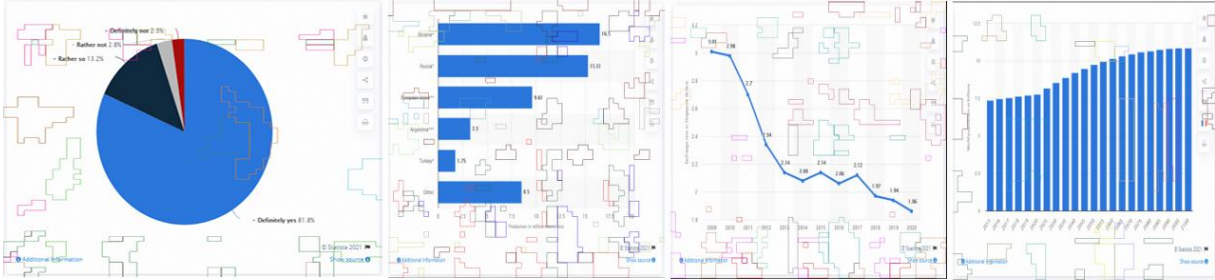
Nghiên cứu về **Visual Token Merging (VTM)** (Bảng 3) là bằng chứng then chốt. Khi chạy ở độ phân giải 768x768, việc *không* sử dụng VTM sẽ gây lỗi **OOM** (Hết bộ nhớ). Bằng cách sử dụng VTM (với $r=84$), mô hình có thể chạy được, giảm độ dài chuỗi từ 2916 xuống 732, và đạt hiệu suất cao nhất. Tác giả cũng trực quan hóa (Hình 5) cho thấy VTM gộp hiệu quả các vùng nền và mảng màu lớn, giữ lại các chi tiết quan trọng.

Model	Setting	ChartQA		
		Cal. (761)	Non-cal. (1739)	Total (2500)
TinyChart@768	Direct	56.64	84.99	76.36
TinyChart@768	PoT	78.98	81.66	80.84
TinyChart@768	Combine	80.42	84.99	83.60

Bảng 2: Hiệu suất ChartQA trên các câu hỏi Tính toán (Cal.) và Không tính toán (Non cal.).

Row	Resolution	GPT PoT	Template PoT	Visual Patch Merge	Visual Length	Inference Throughput	ChartQA			Chart2Text	Chart2Table
							Direct	PoT	Combine	BLEU4	RMS _{F1}
1	384×384	×	×	×	729	3.73 it/s	70.72	-	-	17.10	85.80
2	384×384	×	✓	×	729	3.73 it/s	71.12	55.44	73.00	17.04	87.68
3	384×384	✓	✓	×	729	3.73 it/s	72.44	76.88	79.48	16.67	87.30
4	512×512	✓	✓	×	1,296	2.38 it/s	74.08	79.64	81.72	17.32	89.76
5	512×512	✓	✓	$r=12$	984	2.84 it/s	73.24	77.72	80.52	16.54	88.26
6	512×512	✓	✓	$r=15$	906	3.26 it/s	72.52	78.60	80.04	16.96	88.01
7	512×512	✓	✓	$r=20$	776	3.65 it/s	73.36	78.84	80.76	16.57	87.81
8	768×768	✓	✓	×	2,916	OOM	-	-	-	-	-
9	768×768	✓	✓	$r=84$	732	3.14 it/s	73.24	77.72	81.04	16.43	88.90

Bảng 3: Nghiên cứu loại bỏ. Chúng tôi chỉ huấn luyện các mô hình sử dụng các bộ dữ liệu chuẩn trong thí nghiệm này



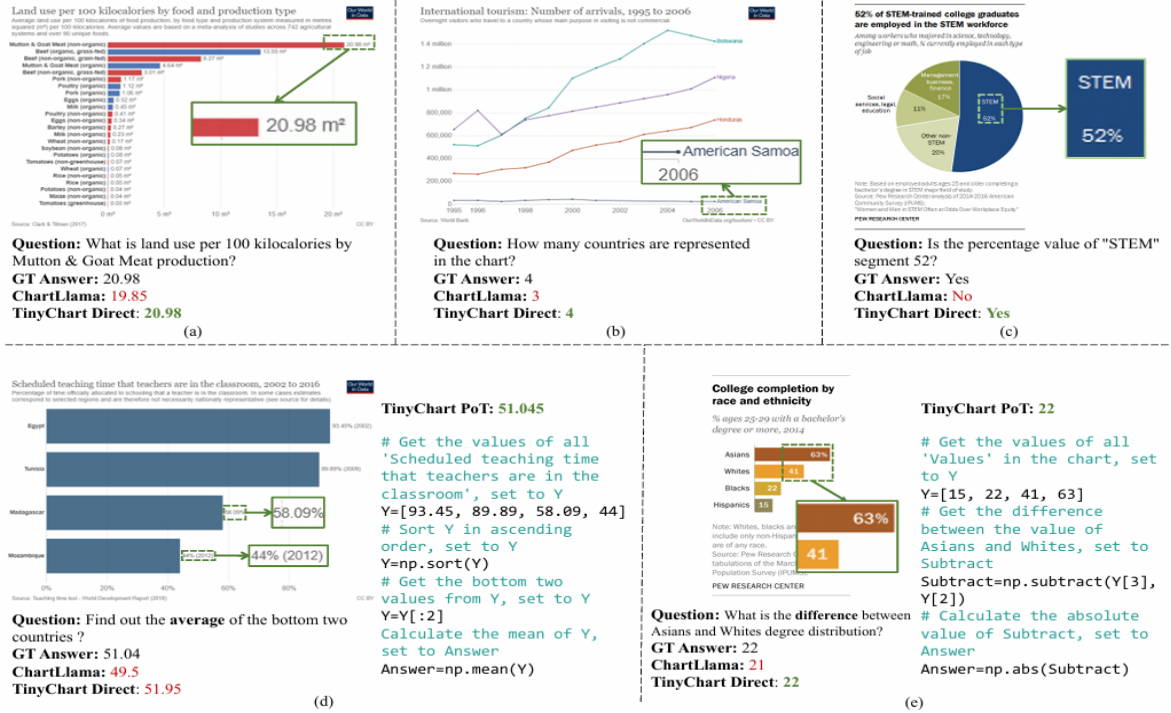
Hình 5: Minh họa việc gộp các token trực quan. 10 nhóm có số token được gộp nhiều nhất được đánh dấu bằng các màu khác nhau.

3.5.4. Nghiên cứu Tình huống và Hạn chế

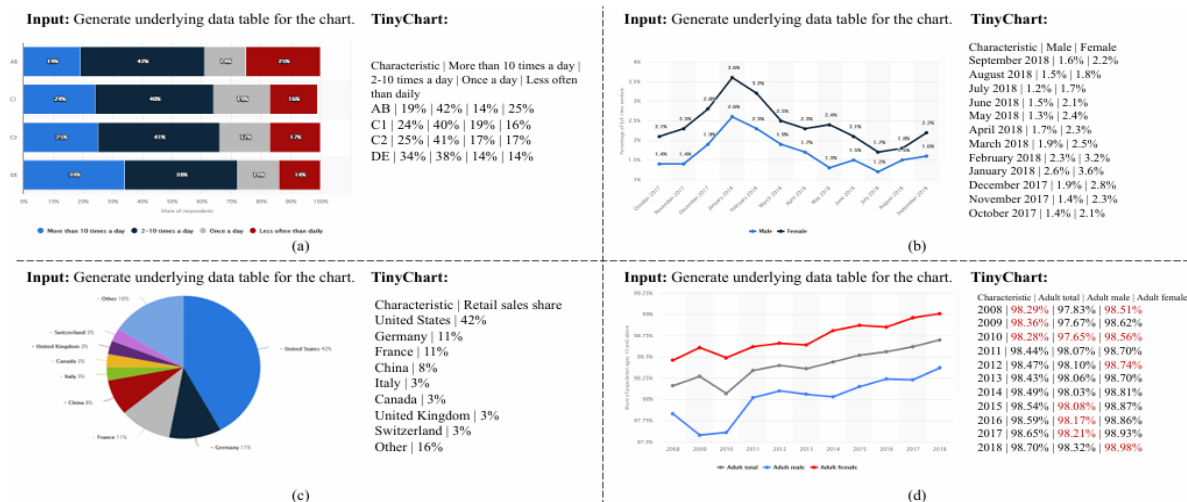
Cuối cùng, các nghiên cứu tình huống (Mục 4.6 trong bài báo, Hình 6-9) cung cấp cái nhìn định tính.

- **Ưu điểm:** Nhờ VTM (ảnh nét), TinyChart đọc đúng các số liệu OCR (ví dụ: "20.98 m²") mà ChartLlama (ảnh mờ) đọc sai. Nhờ PoT, TinyChart tính toán đúng các phép logic (ví dụ: tìm trung bình của 2 giá trị nhỏ nhất) mà mô hình khác tính sai.
- **Hạn chế:** Bài báo cũng chỉ ra các hạn chế còn tồn tại. Mô hình vẫn gặp khó khăn khi phải *ước lượng* giá trị trên trục tọa độ (nếu không có số liệu OCR rõ ràng). Nó vẫn có thể

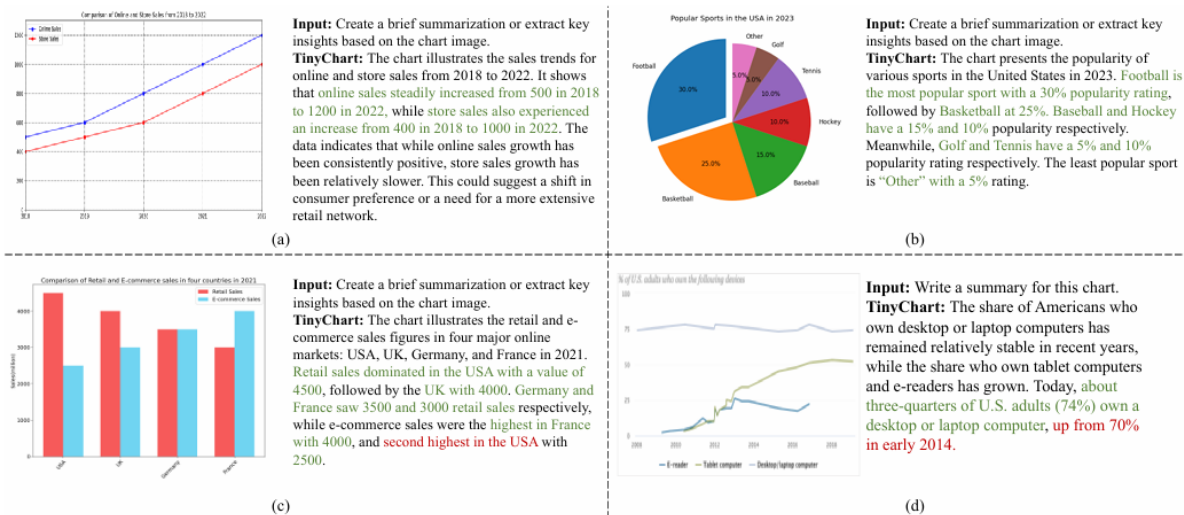
bị "ảo giác" (hallucination), tạo ra thông tin sai lệch (Hình 8). Đồng thời, mô hình không thể vẽ lại các loại biểu đồ phức tạp (như 3D) mà nó chưa từng thấy trong dữ liệu huấn luyện (Hình 9).



Hình 6: Các nghiên cứu điển hình về ChartQA. Chúng tôi so sánh TinyChart@768 với ChartLlama.

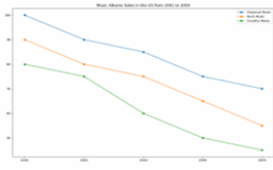


Hình 7: Ví dụ về việc chuyển đổi từ biểu đồ sang bảng của TinyChart@768. Các giá trị sai do mô hình tạo ra được đánh dấu màu đỏ

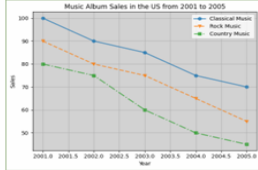


Hình 8: Các trường hợp tạo văn bản từ biểu đồ bởi TinyChart@768. Nội dung đúng được hiển thị bằng màu xanh lá và nội dung sai được đánh dấu bằng màu đỏ.

Input: Redraw the chart with Python code.



TinyChart:

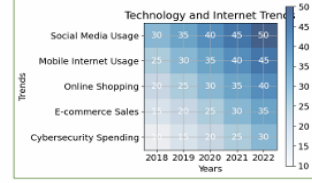


(a)

Input: Redraw the chart with Python code.



TinyChart:



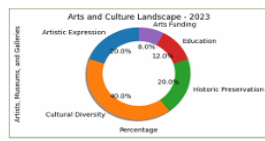
(b)

Input: Redraw the chart with Python code.

Arts and Culture Landscape - 2023



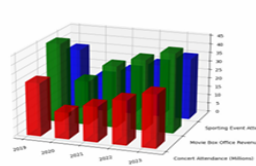
TinyChart:



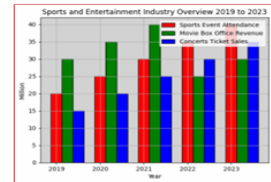
(c)

Input: Redraw the chart with Python code.

Sports and Entertainment Industry Overview - 2019 to 2023



TinyChart:



(d)

Hình 9: Ví dụ về việc vẽ lại biểu đồ. Chúng tôi trình bày hình ảnh kết quả sau khi thực thi mã Python do mô hình tạo ra. Trường hợp xấu là với hộp giới hạn màu đỏ.